instituto superior de engenharia de lisboa

Departamento de Engenharia de Electrónica e Telecomunicações e de Computadores

Segundo Trabalho Prático

51777 : Bernardo Charneca (a51777@alunos.isel.pt)

51744 : Daniel Barreiro (a51744@alunos.isel.pt)

51447 : Ricardo Pinto (a51447@alunos.isel.pt)

Relatório para a Unidade Curricular de Lógica e Computação
da Licenciatura em Engenharia Informática e de Computadores

Professor : Dr. Nuno Leite

29 – dezembro – 2025

Resumo

Este trabalho tem como objetivos: compreender e masterizar a sintaxe dos programas PROLOG; compreender e controlar *backtracking*; saber como usar predicados *built-in* típicos; compreender como aplicar *Constraint Logic Programming* (CLP) e perceber o que é o *Constraint Satisfaction Problem* e aplicar CLP na resolução do problema. Para atingir os objetivos propostos, foi realizada a implementação de uma programa para resolver o *Examination Timetabling Problem* em PROLOG.

Resume

This work has the following objectives: understand and master the syntax of PROLOG programs; understand and control backtracking; know how to use typical built-in predicates; understand how to apply Constraint Logic Programming (CLP); understand what is a Constraint Satisfaction Problem and apply CLP to solve it; To achieve the proposed objectives, an implementation of a program to solve the Examination Timetabling Problem was carried out in PROLOG.

Índice

1 Examination Timetabling Problem **1**

1.1 Explicação dos ficheiros `PROGRAM.names` 1

1.2 Explicação dos predicados do ficheiro `data_loading.pl` 1

1.3 Explicação dos predicados do ficheiro `model.pl` 2

1.4 Explicação dos predicados do ficheiro `soft_constraints.pl` 3

1.5 Explicação do predicado do ficheiro `optimization.pl` 4

1.6 Explicação dos predicados do ficheiro `output.pl` 4

1.7 Explicação do ficheiro `main.pl` 5

1.8 Instruções 5

1.9 Considerações finais 6

1

Examination Timetabling Problem

1.1 Explicação dos ficheiros `PROGRAM.names`

Os ficheiros `PROGRAM.names` são uma coleção de 5 ficheiros (leetc.names, leic.names, lercm.names, meet.names e meic.names), cujo cada ficheiro é organizado em 3 colunas separadas pelo caractere `','`. A primeira coluna representa o nome da disciplina, a segunda coluna representa o semestre em que a disciplina é lecionada e a terceira coluna representa o ano em que a disciplina é lecionada. PROGRAM representa o curso a que corresponde o ficheiro. Por exemplo, o ficheiro `leic.names` corresponde ao curso de Licenciatura em Engenharia Informática e de Computadores.

`Nota: Entende-se por PROGRAM como podendo ser: leetc, leic, lercm, meet ou meic.`

1.2 Explicação dos predicados do ficheiro `data_loading.pl`

Os predicados do ficheiro `data_loading.pl` são responsáveis por carregar os dados dos ficheiros da DEETC *dataset* (deetc0910_1stEpoch.cm, deetc0910_1stEpoch.crs, deetc0910_1stEpoch.names, deetc0910_2ndEpoch.cm, deetc0910_2ndEpoch.crs e deetc0910_2ndEpoch.names) e dos ficheiros `PROGRAM.names`, que já foram explicados anteriormente. Os dados carregados são armazenados sob a forma de factos dinâmicos, permitindo a sua utilização posterior pelo solver. Os predicados são as seguintes:

`load_courses_names/2`

Predicado que carrega o nome das disciplinas de um ficheiro `deetc0910_EPOCH.names`. Cada disciplina é associada a um índice interno. Se o ficheiro for carregado com sucesso, o predicado retorna *true*.

`load_names_lines/3`

Predicado auxiliar para o predicado load_courses_names/2 que carrega os nomes linha por linha.

load_number_of_courses/1

Predicado que carrega o número de cursos a partir de um ficheiro deetc0910_EPOCH.crs. Retorna *true* se o ficheiro for carregado com sucesso.

load_conflict_matrix/2

Predicado que carrega a matriz de conflitos a partir de um ficheiro deetc0910_EPOCH.cm. Retorna *true* se o ficheiro for carregado com sucesso.

load_conflict_matrix_rows/3

Predicado auxiliar para o predicado load_conflict_matrix/2 que carrega as linhas da matriz de conflitos linha por linha.

split_row/2

Predicado auxiliar para o predicado load_conflict_matrix_rows/3 que divide uma linha da matriz de conflitos em números.

load_conflict_cells/4

Predicado auxiliar para o predicado load_conflict_matrix_rows/3 que carrega células individuais de uma linha da matriz de conflitos.

load_programme/2

Predicado que carrega os dados das disciplinas de um curso. Cada disciplina é associada ao respetivo semestre e ano curricular. Retorna *true* se o ficheiro for carregado com sucesso.

load_deetc/0

Predicado que carrega todos os ficheiros de dados da DEETC. Retorna *true* se todos os ficheiros forem carregados com sucesso. Nota: Entende-se por EPOCH como podendo ser: 1stEpoch ou 2ndEpoch. PROGRAM continua com a definição anterior.

1.3 Explicação dos predicados do ficheiro model.pl

Os predicados do ficheiro model.pl são responsáveis por definir o modelo do *Examination Timetabling Problem*. Os predicados são os seguintes:

create_exam_times/2

Predicado que cria as variáveis de decisão que representam os tempos dos exames.

`h1_constraints/2`

Predicado que implementa a restrição `H1` (Hard) - *`There cannot exist students sitting for more than one exam simultaneously`*. funciona tanto para a primeira época como para a segunda época.

`no_name_slot/2`

Predicado auxiliar para o predicado `h1_constraints/2` que verifica se dois exames conflictuantes (com alunos em comum) não estão marcados para o mesmo slot.

`symmetry_breaking/2`

Assegura que o primeiro exame da primeira época é sempre atribuído ao primeiro slot. Como os slots da segunda época estão restritos ao intervalo [17..28], o primeiro exame da segunda época é implicitamente colocado após o último slot da primeira época.

`distance_window/3`

Predicado que calcula a janela mínima e máxima de dias permitidos entre os exames de primeira e segunda época de uma mesma disciplina, de acordo com o dia do exame da primeira época

`second_epoch/2`

Predicado que verifica se dois exames são da mesma disciplina em épocas diferentes.

`h2_constraints/2`

Predicado que implementa a restrição `H2` (Hard)- *`A minimum distance between the two exams of a course must be observed`*.

1.4 Explicação dos predicados do ficheiro `soft_constraints.pl`

Os predicados do ficheiro `soft_constraints.pl` são responsáveis por definir as restrições suaves do *Examination Timetabling Problem*, em particular a restrição `S1`. Os predicados são os seguintes:

`pair_penalty/5`

Predicado que calcula a penalização associada a um par de exames de disciplinas com alunos em comum, tendo em conta a distância temporal entre os respetivos slots e o número de estudantes em conflito.

`proximity_weight/2`

Predicado auxiliar que calcula o peso de proximidade entre dois exames baseado na distância entre os seus slots, com base na distância entre os seus slots, de acordo com uma função de proximidade predefinida.

`s1_cost/3`

Predicado que calcula o custo total das penalizações resultantes de todos os pares de exames com alunos em comum para uma dada época. Responsável pela implementação da restrição `S1` (Soft)– *`Exams should be spread out evenly through the timetable, according to a given proximity function`*.

1.5 Explicação do predicado do ficheiro `optimization.pl`

O predicado do ficheiro `optimization.pl` é responsável por minimizar o custo total do *Examination Timetabling Problem*, através do uso de técnicas de pesquisa com otimização fornecidas pela biblioteca `clpfd`. O predicado é o seguinte:

`minimize_schedule/3`

O predicado realiza a atribuição dos valores às variáveis de decisão utilizando o predicado `labeling/2` com a opção `min/1`, garantindo que a solução encontrada corresponde ao menor custo total possível. `Nota: Este predicado recebe como parâmetro TotalCost (custo total) sendo que, no predicado principal TotalCost, é calculado previamente.`

1.6 Explicação dos predicados do ficheiro `output.pl`

Os predicados do ficheiro `output.pl` são responsáveis por apresentar a solução do *Examination Timetabling Problem* de forma legível, em ficheiros *`Comma Separated Values`* `(.csv)`. Os predicados são os seguintes:

`write_all_programme_csvs/2`

Predicado que escreve um ficheiro `.csv` para cada disponível, contendo os horários das duas épocas de exame.

`write_programme_csv/3`

Predicado auxiliar que escreve o ficheiro `.csv` para um curso específico.

`write_epoch_programme_csv/4`

Predicado auxiliar que escreve os dados de uma época de exames para um curso específico.

`all_programmes/1`

Predicado auxiliar que recolhe todos os cursos disponíveis.

`write_combined_csv/3`

Predicado que escreve um único ficheiro `.csv` combinado com os horários para todos os cursos.

`write_combined_epoch_csv/3`

Predicado auxiliar que escreve os dados de uma época de exames para todos os cursos.

1.7 Explicação do ficheiro `main.pl`

o ficheiro `main.pl` é o ponto principal de execução do programa. Contém o predicado `main/0`, que orquestra o processo de resolução do *Examination Timetabling Problem* chamando os predicados apropriados dos diferentes ficheiros, por uma ordem sequencial lógica.

A sequência de execução é a seguinte:

1. **Carregar os dados**: através do predicado `load_deetc/0`, que importa os ficheiros de dados da DEETC e dos cursos.
2. **Criar variáveis de decisão**: com `create_exam_times/2`, que gera os slots de exames para a primeira e segunda época.
3. **Aplicar restrições hard**: com os predicados `h1_constraints/2` e `h2_constraints/2`.
4. **Calcular custo soft**: com `s1_cost/3`, que avalia a penalização por proximidade de exames de disciplinas com alunos em comum.
5. **Minimizar o custo total**: com `minimize_schedule/3` utilizando técnicas de otimização do `CLP(FD)`.
6. **Gerar output**: com `write_all_programme_csvs/2` e `write_combined_csv/3`, que produzem ficheiros CSV individuais para cada curso e um ficheiro combinado.
7. **Mostrar o custo total**: imprime no terminal o valor de `TotalCost` correspondente à solução encontrada.

Nota: As variáveis `Times1` e `Times2` representam os slots de exames da primeira e segunda época, respetivamente.

1.8 Instruções

Para executar o programa, siga os seguintes passos:

1. Certifique-se de que tem o SWI-Prolog instalado no seu sistema.
2. Navegue até à diretoria onde o ficheiro `main.pl` está localizado (project/)
3. Abra o terminal na diretoria `project/` e digite `swipl main.pl`.
4. Após aparecer `'?-'`, execute o predicado principal com o comando: `main.` **Nota**: o '.' no final do comando é obrigatório.
5. Aguarde enquanto o programa processa os dados e gera os horários de exame.
6. Após a execução, aparece a mensagem `true` indicando que os horários foram gerados com sucesso.

1.9 Considerações finais

O modelo implementado para o *Examination Timetabling Problem* neste projeto gera uma e uma única solução que satisfaz todas as restrições hard definidas (H1 e H2) e minimiza o custo total, sendo ela a melhor solução possível considerando todas as restrições impostas. Fazendo um balanço final sobre o desempenho do modelo, podemos concluir que ele é eficaz na resolução do problema e que conseguimos adquirir os conhecimentos necessários e concluir os objetivos propostos para este projeto.